**Module 2**

**OpenStack Cluster – The Cloud Controller and Common Services**

**Understanding the art of clustering**

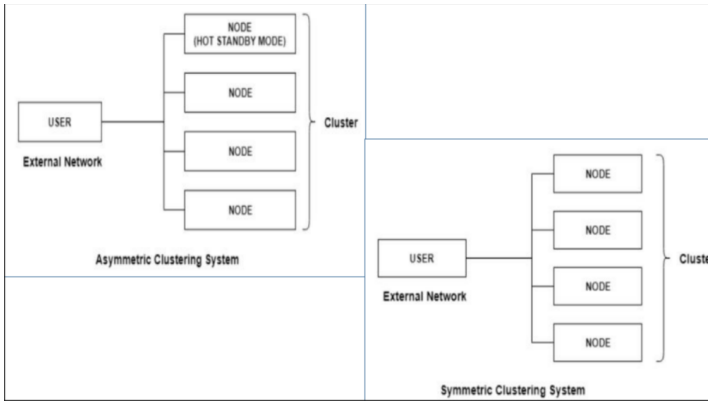
Clustering provides high availability in a given infrastructure. A server cluster is the aggregation of the capacity of two or more servers. This aggregation will be performed by means of the accumulation of several machines. Scaling up (vertical scaling) - refers to adding more expensive and robust servers with more CPU and RAM. scaling down (horizontal scaling)- refers to adding more commodity servers.

**Asymmetric clustering**

Used for high availability purposes and scalability of read/write operations in databases, messaging systems, or files. (Scalability - ability to increase or decrease resources as needed to meet changing demand). A standby server (passive server or sleepy watcher) is involved to take over only if the other server is facing an event of failure.

**Symmetric clustering**

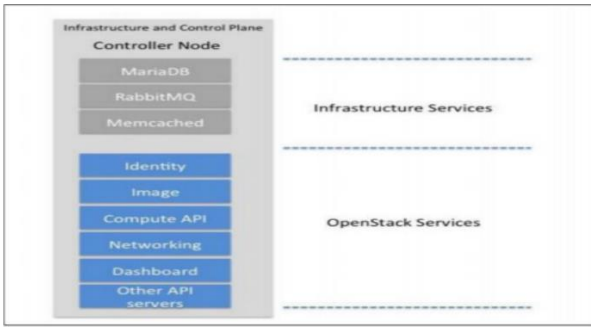
All nodes are active and all participators handle the process of requests. Cost-effective by serving active applications and users. A failed node can be discarded from the cluster, while others take over its workload and continue to handle transactions. Similar to a load-balancing cluster situation, where all nodes share the workload by increasing the performance and scalability of services running in the cloud infrastructure.




**The cloud controller**

The concept of cloud controllers aims to provide central management and control over your OpenStack deployments. Example: cloud controller is managing all API calls and messaging transactions. A cloud controller mainly handles:

1. It presents a gateway for access to cloud management and services consumption.
2. It provides the API services in order to make different OpenStack components talk to each other and provides a service interface to the end user.
3. It provides mechanisms for highly available integrated services by the means of clustering and load-balancing utilities.
4. It provides critical infrastructure services, such as a database and message queue.
5. It exposes the persistent storage, which might be backed onto separate storage nodes.
6. The controller node aggregates the most critical services for OpenStack. The services on the controller node:




**The Keystone service**

The Keystone service provides identity and service cataloging in OpenStack. The Keystone service itself is composed of multiple providers that work in conjunction with each other. Each of these providers implements a concept in the Keystone architecture:

1. Identity
2. Resource
3. Authorization
4. Token
5. Catalog
6. Policy

**The identity provider**

The identity provider validates user and group credentials. OpenStack Keystone provides a built-in identity provider that can be used to create and manage user and group credentials. Keystone can also integrate with external identity providers such as LDAP (Lightweight Directory Access Protocol). Keystone supports various user types to manage access levels to OpenStack services. The user can be one of the following:

A service user who is associated with a service running in OpenStack.
An administrative user who has administrative access to services and resources created.
An end user who has no extra access rights and is a consumer of OpenStack resources.

**The resource provider**

The resource provider provides resources to projects and a domain in Keystone. A domain can be a company or a service provider.

**The authorization provider**

The concept of authorization provides a relationship between OpenStack users and groups to a list of roles. The roles are used to manage access to services running in OpenStack. The Keystone policy provider enforces rules based on the group and role a user belongs to.

**The token provider**

To access the services in OpenStack, the user must authenticate with the identity provider. Once the authentication of the user is established, the token provider generates a token that provides the user with authorization to access the OpenStack services. The token is valid for a limited period of time before it expires. A valid token is required to access the OpenStack services.

**The catalog provider**

All the OpenStack services must register with the Keystone service. The catalog provider maintains a registry of services and associated endpoints running in the OpenStack cloud.

**The policy provider**

The policy provider associates rules to allow access to the Keystone resources. The policies are composed of individual rules that define which users and roles are allowed to access which resources. For example, the ability to add users to a group is provided only to users with an admin role.

**The advanced features provided by the Keystone service:**

Federated Keystone
Fernet tokens

**Federated Keystone**

Federated Identity is a mechanism to use the identity service provided by an external Identity Provider (IdP) to access resources available with the Service Providers (SP). In the case of OpenStack, the identity provider can be a third party while OpenStack acts as the SP with resources such as virtual storage, network, and compute instances. Using Federated Identity has some advantages over maintaining an authentication system in OpenStack:

1. An organization can use an already existing identity source, such as LDAP(Lightweight Directory Access Protocol) or Active Directory, to provide user authentication in OpenStack.
2. It removes the need to manage separate identity sources for OpenStack users.
3. Using an identity service can help in integrating with different cloud services.
4. It provides a single source of truth and user management. So, if a user is part of the identity service, he can be very easily provided access to the OpenStack cloud infrastructure.
5. It lowers the security risk by not running yet another identity system for OpenStack users.
6. It helps in providing better security to OpenStack users and a single sign on.

**How does Federated Identity work?**

To understand this, let's look at how the user interacts with the SP and the Identity Provider services. The steps involved in authenticating users using Federated Identity are as follows:

The user tries to access a resource available with the SP. The SP checks to see whether the user already has an authenticated session. If the user does not have a session, he is redirected to the authentication URL of the Identity Provider. The Identity Provider prompts the user for the identity credentials such as username and password, which it validates, and issues an unscoped token. The unscoped token contains, among other information, the list of groups the authenticated user belongs to. The user then uses the unscoped token to determine a list of accessible domains and projects in the Service Provider's cloud. The user then uses the unscoped token to get a scoped token for the project and domain of his interest and can start using resources on the cloud.

**Fernet tokens**

Traditionally, Keystone issues Public Key Infrastructure (PKI) -based tokens to authenticated users. The user accesses various OpenStack services using these tokens. The PKI tokens encapsulate the user's identity and the authorization context in JSON format. This makes the PKI tokens large in size. In addition, these tokens need to be stored in the database to allow token revocation. PKIZ tokens are an enhancement to the PKI tokens in that they use compression to reduce the size of the tokens but still create considerable stress on the database for holding all the issued tokens. The Fernet tokens are comparatively smaller in size. The major advantage of using Fernet tokens is that they need not be stored in the database and they are considerably faster in token creation as there is no database interaction involved. The Fernet token contains the user identity, scope of authorization over a

project, and an expiration time. Encrypting the identity and authorization with keys generates Fernet tokens. They are considerably smaller and faster than the PKI tokens.

To enable Fernet tokens in Keystone, change the token provider as follows:
keystone.token.providers.fernet.Provider

As Fernet uses encryption keys to generate new tokens, to maintain security of the system these keys need to be rotated from time to time. A Keystone service using the Fernet provider will have multiple Fernet keys active at the same time. The key rotation mechanism is configurable and it determines the number of active keys and frequency of rotation.

**The nova-conductor service**

A vulnerable compute node running the nova-compute service may be attacked and compromised. The job of nova-conductor is to carry out database operation on behalf of compute nodes and provide a layer of database access segregation. It provides security by decoupling direct database access from the compute nodes. nova-conductor compiles a new layer on top of nova-compute. Instead of resolving the complexity of the database requests bottleneck, nova-conductor parallelizes the requests from compute nodes.

**The nova-scheduler service**

OpenStack supports a set of filters that implement a resource availability check on the compute nodes and then runs the filtered list of compute nodes through a weighting mechanism to arrive at the best node to start a virtual machine. The scheduler gives you the choice to configure its options based on a certain number of metrics and policy considerations. nova-scheduler can be thought of as the decision-maker box in a cloud controller node that applies complex algorithms for the efficient usage and placement of virtual machines. The scheduler in OpenStack will be running in the cloud controller node. Other OpenStack services also implement schedulers. For example, cinder-scheduler is a scheduling service for block storage volumes in OpenStack. Similarly, Neutron implements a scheduler to distribute network elements such as routers and DHCP servers among the network nodes.

**The API services**

nova-api is assembled in the controller node. The nova-api service may also fulfill more complicated requests by passing messages within other daemons by means of writing to the databases and queuing messages. As this service is based on the endpoint concept where all API queries are initiated, nova-api provides two different APIs using either the OpenStack API or EC2 API. There is some heterogeneity in the information presentation used by each API; for example, OpenStack uses names and numbers to refer to instances, whereas the EC2 API uses identifiers based on hexadecimal values.

**Image management**

The cloud controller will also be hosting the Glance service for image management that is responsible for the storage, listing, and retrieval of images using the glance-api and glance-registry. The Glance API provides an external REST interface to query virtual machine images and associated metadata. The Glance registry stores the image metadata in the database while the storage backend stores the actual image.

**The network service**

By allowing network's API to reside in the cloud controller, it can support extensions to provide advanced network capabilities, such as firewall, routing, load balancing, and so on.

**The Horizon dashboard**

The OpenStack dashboard runs behind an Apache web server and is based on the Python Django web application framework. Several OpenStack deployments in production run Horizon in the controller node. A separate node that is able to reach the API endpoints to run the Horizon dashboard can be provided to decrease the load on the cloud controller node.

**The telemetry services**

The telemetry service was composed initially of one component, known as Ceilometer, providing metering of resource utilization in OpenStack. Now the telemetry service has been decomposed in two additional projects where it can be optionally enabled to function alongside
**Ceilometer**:
**Aodh**: a sub-project which takes care of generating alarms when resources utilization crosses a certain predefined threshold.
**Gnocchi:** a sub-project for metrics and events storage at scale.
The following are the data collection agents and their roles:

The Ceilometer polling agent provides a plugin framework to collect various resource utilization data. It runs on the controller nodes and provides a flexible means to add data collection agents by implementing them as plugins. The Ceilometer central agent runs on the controller node and uses the REST APIs exposed by other OpenStack services to poll for virtual resources created by tenants. The Central agent can poll for object and block storage, network, and physical hardware using SNMP.

The Ceilometer compute agent runs on the compute node and its main purpose is to collect hypervisor statistics. The Ceilometer IPMI agent runs on the monitored compute node. It polls data by using the Intelligent Platform Management Interface (IPMI) on the servers to collect physical resource utilization data. The monitored servers must be equipped with IPMI sensors and installed with the ipmitool utility. It uses the message bus to send the collected data. The Ceilometer notification agent consumes notification data from various OpenStack components on the message bus. It also runs on the controller node.

**Alarms**

Apart from the various agents for data collection, Ceilometer also contains alarm notification and evaluation agents. The alarm evaluation agent checks for resource utilization parameters crossing a threshold value and uses the notification agent to emit the corresponding notification.

**Events**

Events describe the state change of a resource in OpenStack;

Example: the creation of a volume, starting a virtual machine instance, and so on. Events can be used to provide data to the billing systems. Ceilometer listens to the notification emitted from various OpenStack services and converts them to events.